Open in Colab

Prelab 8.2 Classifying Air-tight Vacuum and Air-leak Vacuum Data using Autoencoders for Anomaly Detection

In [1]:
```
# Let's check the installed tensorflow version
import tensorflow as tf

print('TensorFlow Version is', tf.__version__)
```

```
TensorFlow Version is 2.19.0
```

In [2]:
```
# required Python packages for this colab
!pip install matplotlib
!pip install pandas
!pip install scipy
!pip install scikit-learn
```

```
Requirement already satisfied: matplotlib in /usr/local/lib/python3.12/dist-packages (3.10.0)
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib)
(1.3.3)
Requirement already satisfied: cycler>=0.10 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (0.1
2.1)
Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.12/dist-packages (from matplotlib)
(4.62.1)
Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib)
(1.5.0)
Requirement already satisfied: numpy>=1.23 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (2.0.
2)
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.12/dist-packages (from matplotlib)
(26.0)
Requirement already satisfied: pillow>=8 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (11.3.
0)
Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib)
(3.3.2)
Requirement already satisfied: python-dateutil>=2.7 in /usr/local/lib/python3.12/dist-packages (from matplotl
ib) (2.9.0.post0)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from python-dateutil>=2.7
->matplotlib) (1.17.0)
Requirement already satisfied: pandas in /usr/local/lib/python3.12/dist-packages (2.2.2)
Requirement already satisfied: numpy>=1.26.0 in /usr/local/lib/python3.12/dist-packages (from pandas) (2.0.2)
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.12/dist-packages (from panda
s) (2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.12/dist-packages (from pandas) (2025.2)
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.12/dist-packages (from pandas) (2025.
3)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from python-dateutil>=2.
8.2->pandas) (1.17.0)
Requirement already satisfied: scipy in /usr/local/lib/python3.12/dist-packages (1.16.3)
Requirement already satisfied: numpy<2.6,>=1.25.2 in /usr/local/lib/python3.12/dist-packages (from scipy) (2.
0.2)
Requirement already satisfied: scikit-learn in /usr/local/lib/python3.12/dist-packages (1.6.1)
Requirement already satisfied: numpy>=1.19.5 in /usr/local/lib/python3.12/dist-packages (from scikit-learn)
(2.0.2)
Requirement already satisfied: scipy>=1.6.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn)
(1.16.3)
Requirement already satisfied: joblib>=1.2.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn)
(1.5.3)
Requirement already satisfied: threadpoolctl>=3.1.0 in /usr/local/lib/python3.12/dist-packages (from scikit-l
earn) (3.6.0)
```

Before getting into Lab8, in this prelab, we will first practice a typical machine learning model development hands-on activity. The goal is to create a classification model based on anomaly detection using a machine learning architecture, called autoencoder. The target machine is the vacuum pump system we used in Lab3. The sensor data is the 3-axis acceleration we mainly used in Lab 2 (ADXL345).

The hardware configuration of the vacuum pump system to collect accelerations from the sensor is shown in Figure 3. Sensor placement and axis configuration is illustrated on the top-left side of Figure 3. if you want to see the details of the vacuum pump system and the sensor (ADXL345), please review Lab 2 and Lab 3 manuals. to set up the normal and abnormal conditions, the vacuum valve and the lid cover were manipulated while the vacuum pump is running. In the experiment, the normal condition Figure 4 (left) is defined by air-tight which means firmly closed all valves, connectors, and the lid cover. On the oder hand, the abnormal condition Figure 4 (right) is defined by air-leakage which means opened valves, connectors, or the lid cover. You tell the differences between two conditions in the video clips below.

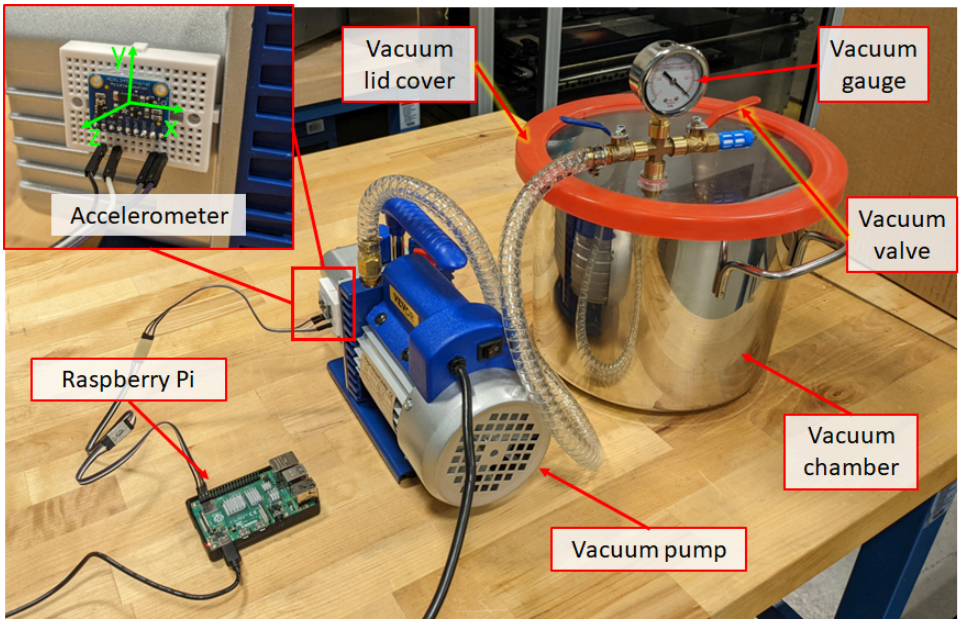


Figure 3 Hardware configuration to collect acceleration of vacuum pump system

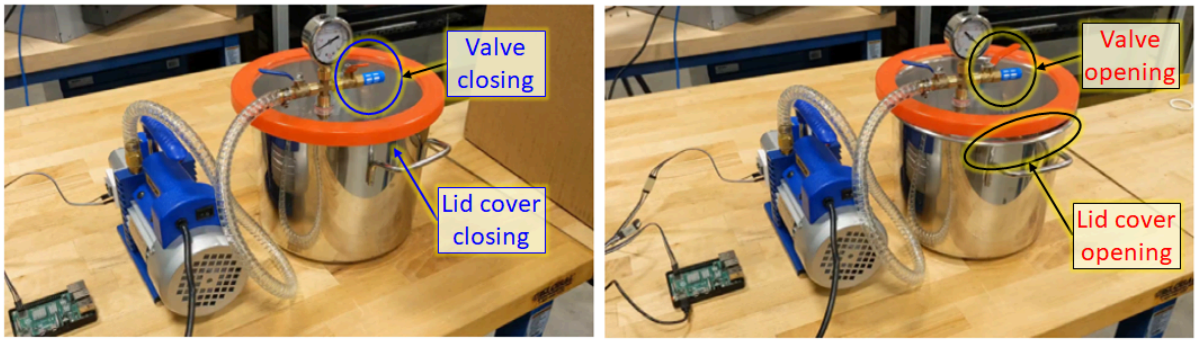


Figure 4 (left) Normal condition (air-tight) and (right) Abnormal condition (air-leakage)

The experiment for data collection is summarized as follows.

- Sensor and axis: ADXL345 (3-axis accelerometer)
- Sampling rate: 1000 Hz (Ts = 0.001 sec.)
- Unit: m/s^2
- Classification (running condition)
  - Normal (air-tight, vacumming) vs. Abnormal (air-leakage, opening)
- Data collection time
  - Normal: 600 seconds (each row: 1 second, total 600 rows)
  - Abnormal: 600 seconds (each row: 1 second, total 600 rows)

This dataset contains 1,200 rows of one second, each with 1000 data points. From the total number of rows, 600 (seconds) rows are labeled from vacuuming and 600 (seconds) of air leakage.

The dataset (prelab8_data.csv, size=40 MB) is as Table 1. In each data cell is equally spaced delimited float array. Each row contains data for 1 second. Because sampling period (Ts) is 1 msec, the number of data points in each cell is 1000. Total number of rows is 1200. On the first column ('Condition') you will see the conditions.

Table 1 Data format: CSV file

Condition	Xacc array [m/s2]	Yacc array [m/s2]	Zacc array [m/s2]
		...	
Vacuuming	$x_1\ x_2 \cdots x_{999}\ x_{1000}$	$y_1\ y_2 \cdots y_{999}\ y_{1000}$	$z_1\ z_2 \cdots z_{999}\ z_{1000}$
Vacuuming	$x_1\ x_2 \cdots x_{999}\ x_{1000}$	$y_1\ y_2 \cdots y_{999}\ y_{1000}$	$z_1\ z_2 \cdots z_{999}\ z_{1000}$
Vacuuming	$x_1\ x_2 \cdots x_{999}\ x_{1000}$	$y_1\ y_2 \cdots y_{999}\ y_{1000}$	$z_1\ z_2 \cdots z_{999}\ z_{1000}$
		...	
Air leakage	$x_1\ x_2 \cdots x_{999}\ x_{1000}$	$y_1\ y_2 \cdots y_{999}\ y_{1000}$	$z_1\ z_2 \cdots z_{999}\ z_{1000}$
Air leakage	$x_1\ x_2 \cdots x_{999}\ x_{1000}$	$y_1\ y_2 \cdots y_{999}\ y_{1000}$	$z_1\ z_2 \cdots z_{999}\ z_{1000}$
Air leakage	$x_1\ x_2 \cdots x_{999}\ x_{1000}$	$y_1\ y_2 \cdots y_{999}\ y_{1000}$	$z_1\ z_2 \cdots z_{999}\ z_{1000}$
		...	

Note: Because this is a labeled dataset, you could phrase this as a supervised learning problem. The goal of this example is to illustrate anomaly detection concepts you can apply to larger datasets, where you do not have labels available (for example, if you had many thousands of normal rhythms, and only a small number of abnormal rhythms).

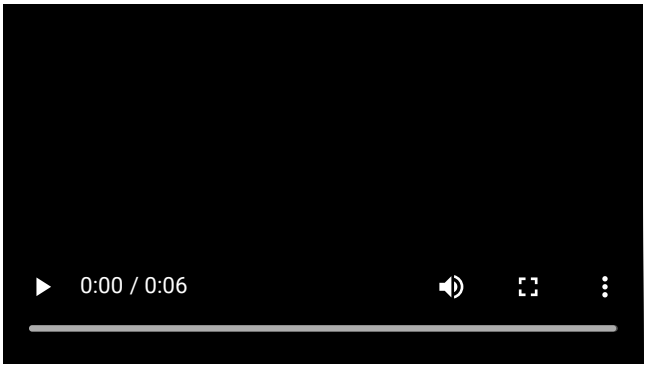
How will you detect anomalies using an autoencoder? An autoencoder is trained to minimize reconstruction error. You will train an autoencoder on the normal vibrations only, then use it to reconstruct all the data. Our hypothesis is that the abnormal vibrations will have higher reconstruction error. You will then classify a vibration as an anomaly if the reconstruction error surpasses a fixed threshold.

Note: This Colab is adapted from the tensorflow Intro to Autoencoders example.

Video clip 1: Normal (vacuumming) condition

In [ ]:
```
# You don't need to run this code block.
# Just play the movie clip on the output cell below.
from IPython.display import HTML
from base64 import b64encode
mp4 = open('Prelab8_Video1_Air-tight.mp4','rb').read()
data_url = "data:video/mp4;base64," + b64encode(mp4).decode()

HTML("""
<video width=400 controls>
      <source src="%s" type="video/mp4">
</video>
""" % data_url)
```

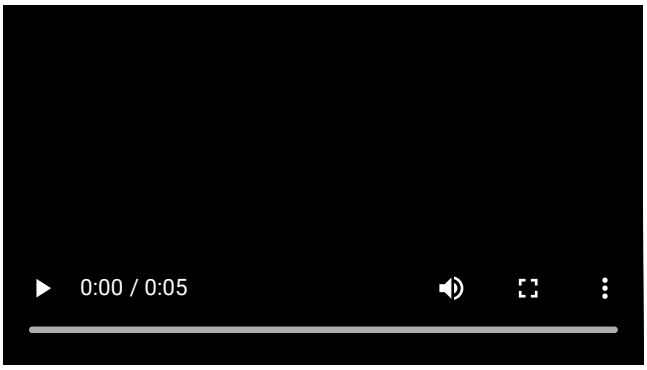
Out[ ]:



Video clip 2: Abnormal (air-leakage) condition

In [ ]:
```
# You don't need to run this code block.
# Just play the movie clip on the output cell below.
from IPython.display import HTML
from base64 import b64encode
mp4 = open('Prelab8_Video2_Air-leakage.mp4','rb').read()
data_url = "data:video/mp4;base64," + b64encode(mp4).decode()

HTML("""
<video width=400 controls>
      <source src="%s" type="video/mp4">
</video>
""" % data_url)
```

Out[ ]:



Working in the X-axis acceleration

Import TensorFlow and other libraries

In [3]:
```
import tensorflow as tf

print('TensorFlow Version is', tf.__version__)
```

```
TensorFlow Version is 2.19.0
```

In [4]:
```
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import scipy.fftpack
from tensorflow import keras

from sklearn.metrics import accuracy_score, precision_score, recall_score, roc_curve, auc
from sklearn.model_selection import train_test_split
from tensorflow.keras import layers, losses
from tensorflow.keras.models import Model
```

Load Vacuum data

The dataset to be used was pulled from a csv filed saved in a github repository prelab8_data.csv.

In [5]:
```
# Copying raw data from github dataset file
url = 'https://raw.githubusercontent.com/purduelamm/purdue_me597_iiot/main/lab/lab8/Prelab8_data.csv'
#df is the variable where the data is stored
df = pd.read_csv(url)
#Display the original content of the dataset
df
```

Out[5]:

	Condition	Xacc array [m/s2]	Yacc array [m/s2]	Zacc array [m/s2]
0	Vacuuming	-4.0011132 -3.4519407999999996 1.2552512 0.0 -...	10.0420096 12.2386992 8.5513988 5.0210048 6.27...	0.6276256 0.0784532 -1.7259703999999998 3.4519...
1	Vacuuming	-0.8629851999999999 -2.6674088 6.5900688 5.021...	11.924886399999998 9.9635564 8.4729456 8.15913...	-4.8640984 -4.0795664 1.569064 4.9425516 1.412...
2	Vacuuming	-1.6475172 -2.1966896 3.7657536 -0.0784532 4.7...	13.1801376 6.5900688 9.1005712 17.1027976 9.41...	-3.138128 3.7657536 0.784532 -0.9414384 -2.510...
3	Vacuuming	-0.9414384 -2.9027684 3.138128 2.4320492 -2.43...	12.552512 5.2563644 10.5127288 11.846433199999...	-2.5105024 -5.099458 3.6088471999999996 1.2552...
4	Vacuuming	-1.8044235999999998 -0.9414384 7.1392411999999...	10.6696352 5.099458 7.139241199999999 15.45528...	-2.6674088 -2.5105024 -1.0198916 -1.2552512 -5...
...	...	...	...	...
1195	Air_leakage	0.6276256 -4.5502856 5.2563644 0.0 -5.0210048 ...	11.2972608 3.6088471999999996 0.5491724 11.924...	0.0 0.6276256 1.0983448 -2.5889556 1.176797999...
1196	Air_leakage	1.0198916 -3.530394 -1.569064 13.72931 -5.6486...	18.1226892 13.1801376 12.160245999999999 1.725...	-6.903881599999999 -2.6674088 -1.8044235999999...
1197	Air_leakage	-1.96133 0.0 3.6873004 0.0 -1.569064 -1.019891...	16.3182656 14.356935599999998 -0.1569064 10.59...	-3.138128 -2.8243152 5.4917240000000005 0.7060...
1198	Air_leakage	4.9425516 -6.1193496 3.7657536 -0.1569064 1.64...	2.5105024 3.7657536 17.8873296 4.6287388 16.63...	-0.7060788 0.9414384 -10.1204628 8.1591328 -6....
1199	Air_leakage	-2.5889556 -4.2364728 -0.0784532 1.72597039999...	9.9635564 15.69064 13.0232312 3.60884719999999...	1.96133 0.0 -4.3933792 4.3933792 1.8828768 -2....

1200 rows × 4 columns

Data Transformation

The strategy to build this model is to select, one by one, data from each axis and assess which axis helps the model classify with better accuracy and precision.

Let's start with the X-axis:

Assign the X-axis on the variable AXIS on the code below.

In [6]:
```
# X-axis: 'Xacc array [m/s2]'
# Y-axis: 'Yacc array [m/s2]'
# Z-axis: 'Zacc array [m/s2]'
# If you want to use x-axis (X-axis),
# AXIS = 'Xacc array [m/s2]'
AXIS = 'Xacc array [m/s2]' #Type the X-axis column name to use data from this axis to build a model    <----

#Exploding the values contained in selected column and converting the string values into float values
df_new = pd.concat([df['Condition'],df[AXIS].str.split(' ', expand=True).astype(float)], axis=1)
ds = df_new.copy()
```

Now, we change the labels 'Vacuuming' and 'Air_leakage' into binary notation for the model usage.

In [7]:
```
#Converting the Classifier into binary values
ds.loc[df['Condition'] == 'Vacuuming', 'Status'] = 1
ds.loc[df['Condition'] == 'Air_leakage', 'Status'] = 0
ds.drop('Condition', axis=1, inplace=True)
#Displays the dataset
ds
```

Out[7]:

	0	1	2	3	4	5	6	7	8	9	...
0	-4.001113	-3.451941	1.255251	0.000000	-2.902768	3.451941	4.550286	3.451941	1.255251	-0.706079	...
1	-0.862985	-2.667409	6.590069	5.021005	-7.217694	-8.237586	-2.588956	-1.255251	-4.471832	-3.451941	...
2	-1.647517	-2.196690	3.765754	-0.078453	4.707192	-1.333704	-0.078453	-1.176798	-0.470719	5.570177	...
3	-0.941438	-2.902768	3.138128	2.432049	-2.432049	-2.353596	-1.098345	5.021005	1.255251	1.333704	...
4	-1.804424	-0.941438	7.139241	-0.706079	0.313813	1.569064	0.078453	1.019892	-0.941438	0.549172	...
...	...	...	...	...	...	...	...	...	...	...	...
1195	0.627626	-4.550286	5.256364	0.000000	-5.021005	1.098345	-9.492837	0.000000	3.295034	3.138128	...
1196	1.019892	-3.530394	-1.569064	13.729310	-5.648630	4.393379	2.745862	-5.021005	5.570177	-1.804424	...
1197	-1.961330	0.000000	3.687300	0.000000	-1.569064	-1.019892	-1.961330	2.196690	-2.667409	2.667409	...
1198	4.942552	-6.119350	3.765754	-0.156906	1.647517	-1.255251	-4.393379	0.235360	6.276256	-6.276256	...
1199	-2.588956	-4.236473	-0.078453	1.725970	-4.707192	-0.706079	-1.882877	-6.903882	13.650857	-12.395606	...

1200 rows × 1001 columns

Now is time to separate the data values from the label values. After that, the data is splitted into training and testing data. A good hereustic number for the test size data is 20% of the dataset.

In [8]:
```
raw_data = ds.values
# The last element contains the labels
labels = raw_data[:, -1]

# The other data points are the vacuum accelerometer data
data = raw_data[:, 0:-1]

train_data, test_data, train_labels, test_labels = train_test_split(
    data, labels, test_size=0.2, random_state=21
)
```

Prior to training the autoencoder, we first apply a min/max scaling transform to the input data which converts it from its original range to a range of (0 to 1) This is done for two main reasons. First, existing research suggests that neural networks in general train better when input values lie between 0 and 1 (or have zero mean and unit variance). Secondly, scaling the data supports the learning objective for the autoencoder (minimizing reconstruction error) and makes the results more interpretable.

In general, the range of output values from the autoencoder is dependent on the type of activation function used in the output layer. For example, the tanh activation function outputs values in the range of -1 and 1, sigmoid outputs values in the range of 0 - 1. In the example, we use the sigmoid activation function in the output layer of the autoencoder, allowing us directly compare the transformed input signal to the output data when computing the means square error metric during training. In addition, having both input and output in the same range allows us to visualize the differences that contribute to the anomaly classification.

In [9]:
```
#Normalizing the values of the dataset
min_val = tf.reduce_min(train_data)
max_val = tf.reduce_max(train_data)

train_data = (train_data - min_val) / (max_val - min_val)
test_data = (test_data - min_val) / (max_val - min_val)

train_data = tf.cast(train_data, tf.float32)
test_data = tf.cast(test_data, tf.float32)
```

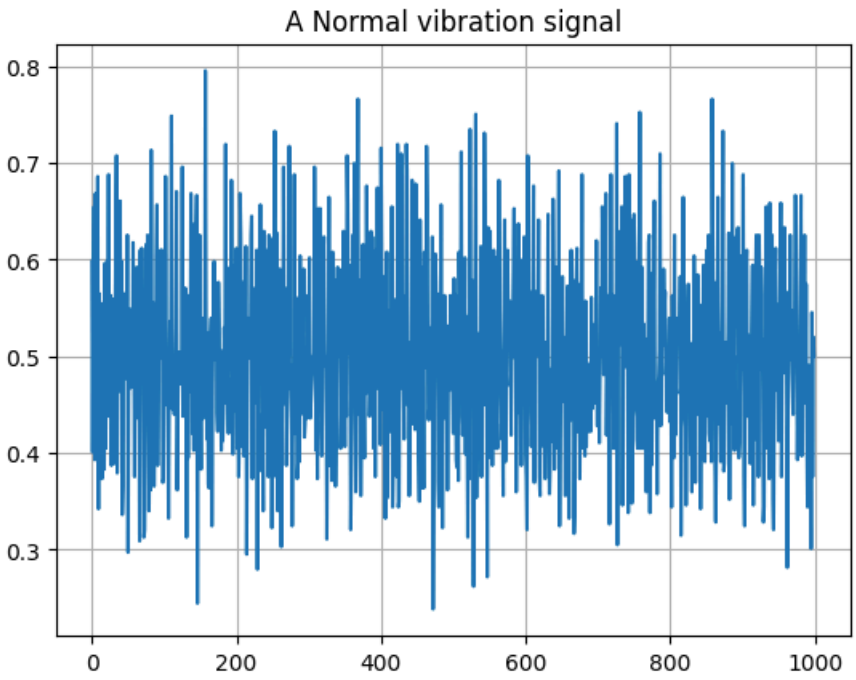
You will train the autoencoder using only the normal rhythms, which are labeled in this dataset as `1`. Separate the normal vibrations from the abnormal vibrations.

In [10]:
```
#Splitting the dataset based on classification: train_labels: Vacuuming, ~train_labels: Air Leakage
train_labels = train_labels.astype(bool)
test_labels = test_labels.astype(bool)

normal_train_data = train_data[train_labels]
normal_test_data = test_data[test_labels]

anomalous_train_data = train_data[~train_labels]
anomalous_test_data = test_data[~test_labels]

portion_of_anomaly_in_training = 0.1 #10% of training data will be anomalies
end_size = int(len(normal_train_data)/(10-portion_of_anomaly_in_training*10))
combined_train_data = np.append(normal_train_data, anomalous_test_data[:end_size], axis=0)
combined_train_data.shape
```

Out[10]:
```
(536, 1000)
```

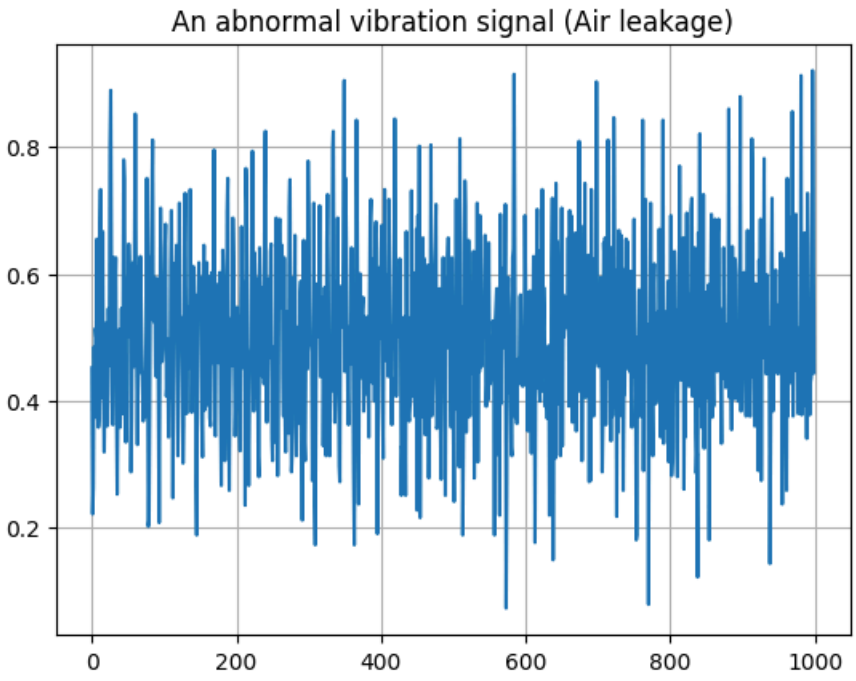
Plot a normal vibration signal

In [11]:
```
#Plotting sample of normal data
plt.grid()
plt.plot(np.arange(1000), normal_train_data[0])
plt.title("A Normal vibration signal")
plt.show()
```



Plot an anomalous vibration signal

In [12]:
```
#Plotting sample of anomalous data
plt.grid()
plt.plot(np.arange(1000), anomalous_train_data[0])
plt.title("An abnormal vibration signal (Air leakage)")
plt.show()
```



Task 2.1

What differences do you notice between the normal and anomalous plots? Can you figure out anomaly based on the signal?

The first thing that I notice when examining both plots is that the mean is higher for the abnormal vibration signal (roughly 0.5) than the normal vibration signal (roughly 0.4). Additionally it looks like the range of values is much larger for the abnormal signal when compared to the normal signal, 0.8 vs 0.5

Picking an Embedding to Build the Model

After training and evaluating the example model, try modifying the size and number of layers to build an understanding for autoencoder architectures.

Note: Changing the size of the embedding (the smallest layer) can produce interesting results. Feel free to play around with that layer size by assigning a value to the variable EMBEDDING_SIZE.

In [67]:
```
#Creating the artificial neural network using Autoencoder
EMBEDDING_SIZE = 10000 #Define how many neurons in the inner layer   <-------------------------------------
class AnomalyDetector(Model):
  def __init__(self):
    super(AnomalyDetector, self).__init__()
    self.encoder = tf.keras.Sequential([
      layers.Dense(256, activation="relu"),
      layers.Dense(128, activation="relu"),
      layers.Dense(EMBEDDING_SIZE, activation="relu")]) # Smallest Layer Defined Here

    self.decoder = tf.keras.Sequential([
      layers.Dense(128, activation="relu"),
      layers.Dense(256, activation="relu"),
      layers.Dense(1000, activation="sigmoid")])

  def call(self, x):
    encoded = self.encoder(x)
    decoded = self.decoder(encoded)
    return decoded
```

```
autoencoder = AnomalyDetector()
print("Chosen Embedding Size: ", EMBEDDING_SIZE)
autoencoder.compile(optimizer='adam', loss='mae')
```

```
Chosen Embedding Size:  10000
```

Train the model

Notice that the autoencoder is trained using only the normal vibrations, but is evaluated using the full test set.

In [68]:
```
#Training the model.
history = autoencoder.fit(normal_train_data, normal_train_data,
          epochs=200,
          batch_size=200,
          validation_data=(test_data, test_data),
          shuffle=True)
```

```
Epoch 1/200
3/3 ━━━━━━━━━━━━━━━━━━━━ 5s 381ms/step - loss: 0.0737 - val_loss: 0.0916
Epoch 2/200
3/3 ━━━━━━━━━━━━━━━━━━━━ 1s 225ms/step - loss: 0.0736 - val_loss: 0.0916
Epoch 3/200
3/3 ━━━━━━━━━━━━━━━━━━━━ 1s 119ms/step - loss: 0.0736 - val_loss: 0.0916
Epoch 4/200
3/3 ━━━━━━━━━━━━━━━━━━━━ 1s 125ms/step - loss: 0.0736 - val_loss: 0.0916
Epoch 5/200
3/3 ━━━━━━━━━━━━━━━━━━━━ 1s 124ms/step - loss: 0.0736 - val_loss: 0.0916
Epoch 6/200
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 116ms/step - loss: 0.0736 - val_loss: 0.0916
Epoch 7/200
3/3 ━━━━━━━━━━━━━━━━━━━━ 1s 122ms/step - loss: 0.0736 - val_loss: 0.0916
Epoch 8/200
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 117ms/step - loss: 0.0736 - val_loss: 0.0916
Epoch 9/200
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 123ms/step - loss: 0.0736 - val_loss: 0.0916
Epoch 10/200
3/3 ━━━━━━━━━━━━━━━━━━━━ 1s 118ms/step - loss: 0.0736 - val_loss: 0.0916
Epoch 11/200
3/3 ━━━━━━━━━━━━━━━━━━━━ 1s 116ms/step - loss: 0.0736 - val_loss: 0.0916
Epoch 12/200
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 128ms/step - loss: 0.0736 - val_loss: 0.0916
Epoch 13/200
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 118ms/step - loss: 0.0735 - val_loss: 0.0916
Epoch 14/200
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 120ms/step - loss: 0.0735 - val_loss: 0.0916
Epoch 15/200
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 135ms/step - loss: 0.0735 - val_loss: 0.0916
Epoch 16/200
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 123ms/step - loss: 0.0735 - val_loss: 0.0916
Epoch 17/200
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 121ms/step - loss: 0.0735 - val_loss: 0.0916
Epoch 18/200
3/3 ━━━━━━━━━━━━━━━━━━━━ 1s 118ms/step - loss: 0.0735 - val_loss: 0.0916
Epoch 19/200
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 121ms/step - loss: 0.0735 - val_loss: 0.0916
Epoch 20/200
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 126ms/step - loss: 0.0735 - val_loss: 0.0916
Epoch 21/200
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 118ms/step - loss: 0.0735 - val_loss: 0.0916
Epoch 22/200
3/3 ━━━━━━━━━━━━━━━━━━━━ 1s 125ms/step - loss: 0.0735 - val_loss: 0.0916
Epoch 23/200
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 122ms/step - loss: 0.0735 - val_loss: 0.0916
Epoch 24/200
3/3 ━━━━━━━━━━━━━━━━━━━━ 1s 220ms/step - loss: 0.0735 - val_loss: 0.0916
Epoch 25/200
3/3 ━━━━━━━━━━━━━━━━━━━━ 1s 229ms/step - loss: 0.0735 - val_loss: 0.0916
Epoch 26/200
3/3 ━━━━━━━━━━━━━━━━━━━━ 1s 193ms/step - loss: 0.0735 - val_loss: 0.0916
Epoch 27/200
3/3 ━━━━━━━━━━━━━━━━━━━━ 1s 196ms/step - loss: 0.0735 - val_loss: 0.0916
Epoch 28/200
3/3 ━━━━━━━━━━━━━━━━━━━━ 1s 186ms/step - loss: 0.0735 - val_loss: 0.0916
Epoch 29/200
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 125ms/step - loss: 0.0735 - val_loss: 0.0916
Epoch 30/200
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 118ms/step - loss: 0.0735 - val_loss: 0.0916
Epoch 31/200
3/3 ━━━━━━━━━━━━━━━━━━━━ 1s 121ms/step - loss: 0.0735 - val_loss: 0.0916
Epoch 32/200
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 114ms/step - loss: 0.0735 - val_loss: 0.0916
Epoch 33/200
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 122ms/step - loss: 0.0735 - val_loss: 0.0916
Epoch 34/200
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 118ms/step - loss: 0.0735 - val_loss: 0.0916
Epoch 35/200
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 125ms/step - loss: 0.0735 - val_loss: 0.0916
```

```
Epoch 36/200
3/3 ━━━━━━━━━━━━━━━━━━━━ 1s 129ms/step - loss: 0.0735 - val_loss: 0.0916
Epoch 37/200
3/3 ━━━━━━━━━━━━━━━━━━━━ 1s 120ms/step - loss: 0.0735 - val_loss: 0.0916
Epoch 38/200
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 129ms/step - loss: 0.0735 - val_loss: 0.0916
Epoch 39/200
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 123ms/step - loss: 0.0735 - val_loss: 0.0916
Epoch 40/200
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 120ms/step - loss: 0.0735 - val_loss: 0.0916
Epoch 41/200
3/3 ━━━━━━━━━━━━━━━━━━━━ 1s 126ms/step - loss: 0.0735 - val_loss: 0.0916
Epoch 42/200
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 119ms/step - loss: 0.0735 - val_loss: 0.0916
Epoch 43/200
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 120ms/step - loss: 0.0735 - val_loss: 0.0916
Epoch 44/200
3/3 ━━━━━━━━━━━━━━━━━━━━ 1s 119ms/step - loss: 0.0735 - val_loss: 0.0916
Epoch 45/200
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 125ms/step - loss: 0.0735 - val_loss: 0.0916
Epoch 46/200
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 126ms/step - loss: 0.0735 - val_loss: 0.0916
Epoch 47/200
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 118ms/step - loss: 0.0735 - val_loss: 0.0916
Epoch 48/200
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 118ms/step - loss: 0.0735 - val_loss: 0.0916
Epoch 49/200
3/3 ━━━━━━━━━━━━━━━━━━━━ 1s 116ms/step - loss: 0.0735 - val_loss: 0.0916
Epoch 50/200
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 120ms/step - loss: 0.0735 - val_loss: 0.0916
Epoch 51/200
3/3 ━━━━━━━━━━━━━━━━━━━━ 1s 185ms/step - loss: 0.0735 - val_loss: 0.0916
Epoch 52/200
3/3 ━━━━━━━━━━━━━━━━━━━━ 1s 206ms/step - loss: 0.0735 - val_loss: 0.0916
Epoch 53/200
3/3 ━━━━━━━━━━━━━━━━━━━━ 1s 193ms/step - loss: 0.0735 - val_loss: 0.0916
Epoch 54/200
3/3 ━━━━━━━━━━━━━━━━━━━━ 1s 219ms/step - loss: 0.0735 - val_loss: 0.0916
Epoch 55/200
3/3 ━━━━━━━━━━━━━━━━━━━━ 1s 168ms/step - loss: 0.0735 - val_loss: 0.0916
Epoch 56/200
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 122ms/step - loss: 0.0735 - val_loss: 0.0916
Epoch 57/200
3/3 ━━━━━━━━━━━━━━━━━━━━ 1s 213ms/step - loss: 0.0735 - val_loss: 0.0916
Epoch 58/200
3/3 ━━━━━━━━━━━━━━━━━━━━ 1s 224ms/step - loss: 0.0735 - val_loss: 0.0916
Epoch 59/200
3/3 ━━━━━━━━━━━━━━━━━━━━ 1s 204ms/step - loss: 0.0735 - val_loss: 0.0916
Epoch 60/200
3/3 ━━━━━━━━━━━━━━━━━━━━ 1s 250ms/step - loss: 0.0735 - val_loss: 0.0916
Epoch 61/200
3/3 ━━━━━━━━━━━━━━━━━━━━ 1s 128ms/step - loss: 0.0735 - val_loss: 0.0916
Epoch 62/200
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 120ms/step - loss: 0.0735 - val_loss: 0.0916
Epoch 63/200
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 127ms/step - loss: 0.0735 - val_loss: 0.0916
Epoch 64/200
3/3 ━━━━━━━━━━━━━━━━━━━━ 1s 127ms/step - loss: 0.0735 - val_loss: 0.0916
Epoch 65/200
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 125ms/step - loss: 0.0735 - val_loss: 0.0916
Epoch 66/200
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 120ms/step - loss: 0.0735 - val_loss: 0.0916
Epoch 67/200
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 123ms/step - loss: 0.0735 - val_loss: 0.0916
Epoch 68/200
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 124ms/step - loss: 0.0735 - val_loss: 0.0916
Epoch 69/200
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 126ms/step - loss: 0.0735 - val_loss: 0.0916
Epoch 70/200
3/3 ━━━━━━━━━━━━━━━━━━━━ 1s 118ms/step - loss: 0.0735 - val_loss: 0.0916
```

```
Epoch 71/200
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 118ms/step - loss: 0.0735 - val_loss: 0.0916
Epoch 72/200
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 118ms/step - loss: 0.0735 - val_loss: 0.0916
Epoch 73/200
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 129ms/step - loss: 0.0735 - val_loss: 0.0916
Epoch 74/200
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 119ms/step - loss: 0.0735 - val_loss: 0.0916
Epoch 75/200
3/3 ━━━━━━━━━━━━━━━━━━━━ 1s 221ms/step - loss: 0.0735 - val_loss: 0.0916
Epoch 76/200
3/3 ━━━━━━━━━━━━━━━━━━━━ 1s 200ms/step - loss: 0.0735 - val_loss: 0.0916
Epoch 77/200
3/3 ━━━━━━━━━━━━━━━━━━━━ 1s 182ms/step - loss: 0.0735 - val_loss: 0.0916
Epoch 78/200
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 126ms/step - loss: 0.0735 - val_loss: 0.0916
Epoch 79/200
3/3 ━━━━━━━━━━━━━━━━━━━━ 1s 182ms/step - loss: 0.0735 - val_loss: 0.0916
Epoch 80/200
3/3 ━━━━━━━━━━━━━━━━━━━━ 1s 222ms/step - loss: 0.0735 - val_loss: 0.0916
Epoch 81/200
3/3 ━━━━━━━━━━━━━━━━━━━━ 1s 200ms/step - loss: 0.0735 - val_loss: 0.0916
Epoch 82/200
3/3 ━━━━━━━━━━━━━━━━━━━━ 1s 231ms/step - loss: 0.0735 - val_loss: 0.0916
Epoch 83/200
3/3 ━━━━━━━━━━━━━━━━━━━━ 1s 189ms/step - loss: 0.0735 - val_loss: 0.0916
Epoch 84/200
3/3 ━━━━━━━━━━━━━━━━━━━━ 1s 481ms/step - loss: 0.0735 - val_loss: 0.0916
Epoch 85/200
3/3 ━━━━━━━━━━━━━━━━━━━━ 1s 200ms/step - loss: 0.0735 - val_loss: 0.0916
Epoch 86/200
3/3 ━━━━━━━━━━━━━━━━━━━━ 1s 207ms/step - loss: 0.0735 - val_loss: 0.0916
Epoch 87/200
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 125ms/step - loss: 0.0735 - val_loss: 0.0916
Epoch 88/200
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 126ms/step - loss: 0.0735 - val_loss: 0.0916
Epoch 89/200
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 121ms/step - loss: 0.0735 - val_loss: 0.0916
Epoch 90/200
3/3 ━━━━━━━━━━━━━━━━━━━━ 1s 124ms/step - loss: 0.0735 - val_loss: 0.0916
Epoch 91/200
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 121ms/step - loss: 0.0735 - val_loss: 0.0916
Epoch 92/200
3/3 ━━━━━━━━━━━━━━━━━━━━ 1s 122ms/step - loss: 0.0735 - val_loss: 0.0916
Epoch 93/200
3/3 ━━━━━━━━━━━━━━━━━━━━ 1s 126ms/step - loss: 0.0735 - val_loss: 0.0916
Epoch 94/200
3/3 ━━━━━━━━━━━━━━━━━━━━ 1s 222ms/step - loss: 0.0735 - val_loss: 0.0916
Epoch 95/200
3/3 ━━━━━━━━━━━━━━━━━━━━ 1s 219ms/step - loss: 0.0735 - val_loss: 0.0916
Epoch 96/200
3/3 ━━━━━━━━━━━━━━━━━━━━ 1s 200ms/step - loss: 0.0735 - val_loss: 0.0916
Epoch 97/200
3/3 ━━━━━━━━━━━━━━━━━━━━ 1s 197ms/step - loss: 0.0735 - val_loss: 0.0916
Epoch 98/200
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 129ms/step - loss: 0.0735 - val_loss: 0.0916
Epoch 99/200
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 128ms/step - loss: 0.0735 - val_loss: 0.0916
Epoch 100/200
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 124ms/step - loss: 0.0735 - val_loss: 0.0916
Epoch 101/200
3/3 ━━━━━━━━━━━━━━━━━━━━ 1s 126ms/step - loss: 0.0735 - val_loss: 0.0916
Epoch 102/200
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 120ms/step - loss: 0.0735 - val_loss: 0.0916
Epoch 103/200
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 131ms/step - loss: 0.0735 - val_loss: 0.0916
Epoch 104/200
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 119ms/step - loss: 0.0735 - val_loss: 0.0916
Epoch 105/200
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 119ms/step - loss: 0.0735 - val_loss: 0.0916
```

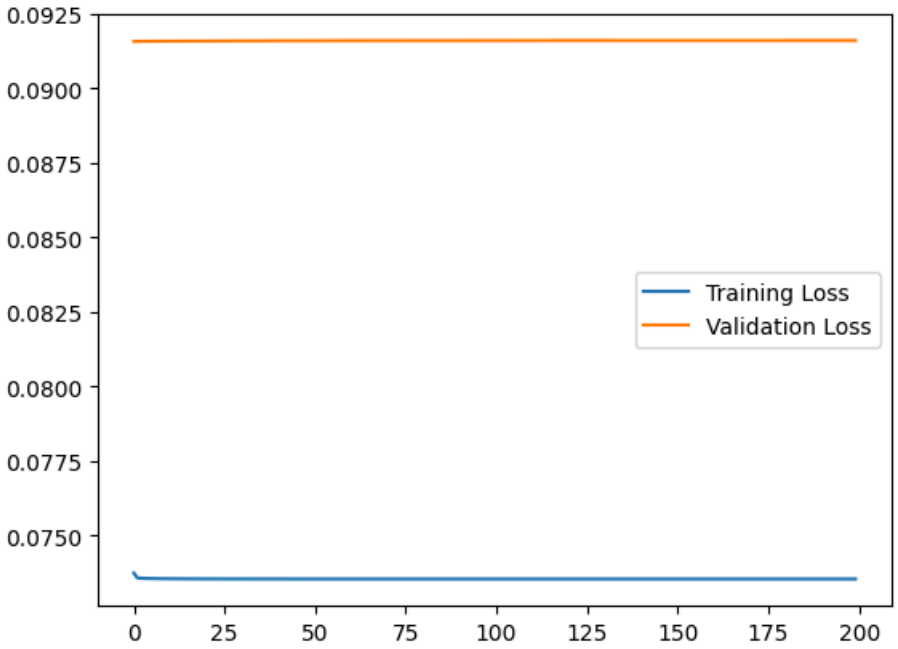
```
Epoch 106/200
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 136ms/step - loss: 0.0735 - val_loss: 0.0916
Epoch 107/200
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 127ms/step - loss: 0.0735 - val_loss: 0.0916
Epoch 108/200
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 121ms/step - loss: 0.0735 - val_loss: 0.0916
Epoch 109/200
3/3 ━━━━━━━━━━━━━━━━━━━━ 1s 119ms/step - loss: 0.0735 - val_loss: 0.0916
Epoch 110/200
3/3 ━━━━━━━━━━━━━━━━━━━━ 1s 129ms/step - loss: 0.0735 - val_loss: 0.0916
Epoch 111/200
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 120ms/step - loss: 0.0735 - val_loss: 0.0916
Epoch 112/200
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 119ms/step - loss: 0.0735 - val_loss: 0.0916
Epoch 113/200
3/3 ━━━━━━━━━━━━━━━━━━━━ 1s 120ms/step - loss: 0.0735 - val_loss: 0.0916
Epoch 114/200
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 128ms/step - loss: 0.0735 - val_loss: 0.0916
Epoch 115/200
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 134ms/step - loss: 0.0735 - val_loss: 0.0916
Epoch 116/200
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 117ms/step - loss: 0.0735 - val_loss: 0.0916
Epoch 117/200
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 121ms/step - loss: 0.0735 - val_loss: 0.0916
Epoch 118/200
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 130ms/step - loss: 0.0735 - val_loss: 0.0916
Epoch 119/200
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 122ms/step - loss: 0.0735 - val_loss: 0.0916
Epoch 120/200
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 121ms/step - loss: 0.0735 - val_loss: 0.0916
Epoch 121/200
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 136ms/step - loss: 0.0735 - val_loss: 0.0916
Epoch 122/200
3/3 ━━━━━━━━━━━━━━━━━━━━ 1s 220ms/step - loss: 0.0735 - val_loss: 0.0916
Epoch 123/200
3/3 ━━━━━━━━━━━━━━━━━━━━ 1s 198ms/step - loss: 0.0735 - val_loss: 0.0916
Epoch 124/200
3/3 ━━━━━━━━━━━━━━━━━━━━ 1s 191ms/step - loss: 0.0735 - val_loss: 0.0916
Epoch 125/200
3/3 ━━━━━━━━━━━━━━━━━━━━ 1s 193ms/step - loss: 0.0735 - val_loss: 0.0916
Epoch 126/200
3/3 ━━━━━━━━━━━━━━━━━━━━ 1s 195ms/step - loss: 0.0735 - val_loss: 0.0916
Epoch 127/200
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 121ms/step - loss: 0.0735 - val_loss: 0.0916
Epoch 128/200
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 121ms/step - loss: 0.0735 - val_loss: 0.0916
Epoch 129/200
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 134ms/step - loss: 0.0735 - val_loss: 0.0916
Epoch 130/200
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 120ms/step - loss: 0.0735 - val_loss: 0.0916
Epoch 131/200
3/3 ━━━━━━━━━━━━━━━━━━━━ 1s 128ms/step - loss: 0.0735 - val_loss: 0.0916
Epoch 132/200
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 121ms/step - loss: 0.0735 - val_loss: 0.0916
Epoch 133/200
3/3 ━━━━━━━━━━━━━━━━━━━━ 1s 125ms/step - loss: 0.0735 - val_loss: 0.0916
Epoch 134/200
3/3 ━━━━━━━━━━━━━━━━━━━━ 1s 125ms/step - loss: 0.0735 - val_loss: 0.0916
Epoch 135/200
3/3 ━━━━━━━━━━━━━━━━━━━━ 1s 134ms/step - loss: 0.0735 - val_loss: 0.0916
Epoch 136/200
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 124ms/step - loss: 0.0735 - val_loss: 0.0916
Epoch 137/200
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 121ms/step - loss: 0.0735 - val_loss: 0.0916
Epoch 138/200
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 124ms/step - loss: 0.0735 - val_loss: 0.0916
Epoch 139/200
3/3 ━━━━━━━━━━━━━━━━━━━━ 1s 128ms/step - loss: 0.0735 - val_loss: 0.0916
Epoch 140/200
3/3 ━━━━━━━━━━━━━━━━━━━━ 1s 122ms/step - loss: 0.0735 - val_loss: 0.0916
```

```
Epoch 141/200
3/3 ━━━━━━━━━━━━━━━━━━━━ 1s 129ms/step - loss: 0.0735 - val_loss: 0.0916
Epoch 142/200
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 127ms/step - loss: 0.0735 - val_loss: 0.0916
Epoch 143/200
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 124ms/step - loss: 0.0735 - val_loss: 0.0916
Epoch 144/200
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 137ms/step - loss: 0.0735 - val_loss: 0.0916
Epoch 145/200
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 119ms/step - loss: 0.0735 - val_loss: 0.0916
Epoch 146/200
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 131ms/step - loss: 0.0735 - val_loss: 0.0916
Epoch 147/200
3/3 ━━━━━━━━━━━━━━━━━━━━ 1s 118ms/step - loss: 0.0735 - val_loss: 0.0916
Epoch 148/200
3/3 ━━━━━━━━━━━━━━━━━━━━ 1s 231ms/step - loss: 0.0735 - val_loss: 0.0916
Epoch 149/200
3/3 ━━━━━━━━━━━━━━━━━━━━ 1s 225ms/step - loss: 0.0735 - val_loss: 0.0916
Epoch 150/200
3/3 ━━━━━━━━━━━━━━━━━━━━ 1s 228ms/step - loss: 0.0735 - val_loss: 0.0916
Epoch 151/200
3/3 ━━━━━━━━━━━━━━━━━━━━ 1s 210ms/step - loss: 0.0735 - val_loss: 0.0916
Epoch 152/200
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 127ms/step - loss: 0.0735 - val_loss: 0.0916
Epoch 153/200
3/3 ━━━━━━━━━━━━━━━━━━━━ 1s 129ms/step - loss: 0.0735 - val_loss: 0.0916
Epoch 154/200
3/3 ━━━━━━━━━━━━━━━━━━━━ 1s 126ms/step - loss: 0.0735 - val_loss: 0.0916
Epoch 155/200
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 129ms/step - loss: 0.0735 - val_loss: 0.0916
Epoch 156/200
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 127ms/step - loss: 0.0735 - val_loss: 0.0916
Epoch 157/200
3/3 ━━━━━━━━━━━━━━━━━━━━ 1s 129ms/step - loss: 0.0735 - val_loss: 0.0916
Epoch 158/200
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 122ms/step - loss: 0.0735 - val_loss: 0.0916
Epoch 159/200
3/3 ━━━━━━━━━━━━━━━━━━━━ 1s 123ms/step - loss: 0.0735 - val_loss: 0.0916
Epoch 160/200
3/3 ━━━━━━━━━━━━━━━━━━━━ 1s 120ms/step - loss: 0.0735 - val_loss: 0.0916
Epoch 161/200
3/3 ━━━━━━━━━━━━━━━━━━━━ 1s 126ms/step - loss: 0.0735 - val_loss: 0.0916
Epoch 162/200
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 125ms/step - loss: 0.0735 - val_loss: 0.0916
Epoch 163/200
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 121ms/step - loss: 0.0735 - val_loss: 0.0916
Epoch 164/200
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 119ms/step - loss: 0.0735 - val_loss: 0.0916
Epoch 165/200
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 132ms/step - loss: 0.0735 - val_loss: 0.0916
Epoch 166/200
3/3 ━━━━━━━━━━━━━━━━━━━━ 1s 135ms/step - loss: 0.0735 - val_loss: 0.0916
Epoch 167/200
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 124ms/step - loss: 0.0735 - val_loss: 0.0916
Epoch 168/200
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 124ms/step - loss: 0.0735 - val_loss: 0.0916
Epoch 169/200
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 139ms/step - loss: 0.0735 - val_loss: 0.0916
Epoch 170/200
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 126ms/step - loss: 0.0735 - val_loss: 0.0916
Epoch 171/200
3/3 ━━━━━━━━━━━━━━━━━━━━ 1s 123ms/step - loss: 0.0735 - val_loss: 0.0916
Epoch 172/200
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 130ms/step - loss: 0.0735 - val_loss: 0.0916
Epoch 173/200
3/3 ━━━━━━━━━━━━━━━━━━━━ 1s 230ms/step - loss: 0.0735 - val_loss: 0.0916
Epoch 174/200
3/3 ━━━━━━━━━━━━━━━━━━━━ 1s 194ms/step - loss: 0.0735 - val_loss: 0.0916
Epoch 175/200
3/3 ━━━━━━━━━━━━━━━━━━━━ 1s 223ms/step - loss: 0.0735 - val_loss: 0.0916
```

```
Epoch 176/200
3/3 ━━━━━━━━━━━━━━━━━━━━ 1s 123ms/step - loss: 0.0735 - val_loss: 0.0916
Epoch 177/200
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 123ms/step - loss: 0.0735 - val_loss: 0.0916
Epoch 178/200
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 127ms/step - loss: 0.0735 - val_loss: 0.0916
Epoch 179/200
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 118ms/step - loss: 0.0735 - val_loss: 0.0916
Epoch 180/200
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 122ms/step - loss: 0.0735 - val_loss: 0.0916
Epoch 181/200
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 131ms/step - loss: 0.0735 - val_loss: 0.0916
Epoch 182/200
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 127ms/step - loss: 0.0735 - val_loss: 0.0916
Epoch 183/200
3/3 ━━━━━━━━━━━━━━━━━━━━ 1s 127ms/step - loss: 0.0735 - val_loss: 0.0916
Epoch 184/200
3/3 ━━━━━━━━━━━━━━━━━━━━ 1s 128ms/step - loss: 0.0735 - val_loss: 0.0916
Epoch 185/200
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 131ms/step - loss: 0.0735 - val_loss: 0.0916
Epoch 186/200
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 117ms/step - loss: 0.0735 - val_loss: 0.0916
Epoch 187/200
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 126ms/step - loss: 0.0735 - val_loss: 0.0916
Epoch 188/200
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 128ms/step - loss: 0.0735 - val_loss: 0.0916
Epoch 189/200
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 118ms/step - loss: 0.0735 - val_loss: 0.0916
Epoch 190/200
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 126ms/step - loss: 0.0735 - val_loss: 0.0916
Epoch 191/200
3/3 ━━━━━━━━━━━━━━━━━━━━ 1s 204ms/step - loss: 0.0735 - val_loss: 0.0916
Epoch 192/200
3/3 ━━━━━━━━━━━━━━━━━━━━ 1s 228ms/step - loss: 0.0735 - val_loss: 0.0916
Epoch 193/200
3/3 ━━━━━━━━━━━━━━━━━━━━ 1s 218ms/step - loss: 0.0735 - val_loss: 0.0916
Epoch 194/200
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 140ms/step - loss: 0.0735 - val_loss: 0.0916
Epoch 195/200
3/3 ━━━━━━━━━━━━━━━━━━━━ 1s 180ms/step - loss: 0.0735 - val_loss: 0.0916
Epoch 196/200
3/3 ━━━━━━━━━━━━━━━━━━━━ 1s 243ms/step - loss: 0.0735 - val_loss: 0.0916
Epoch 197/200
3/3 ━━━━━━━━━━━━━━━━━━━━ 1s 204ms/step - loss: 0.0735 - val_loss: 0.0916
Epoch 198/200
3/3 ━━━━━━━━━━━━━━━━━━━━ 1s 195ms/step - loss: 0.0735 - val_loss: 0.0916
Epoch 199/200
3/3 ━━━━━━━━━━━━━━━━━━━━ 1s 204ms/step - loss: 0.0735 - val_loss: 0.0916
Epoch 200/200
3/3 ━━━━━━━━━━━━━━━━━━━━ 1s 210ms/step - loss: 0.0735 - val_loss: 0.0916
```

In [69]:
```
#Plotting the evolution of training and validation loss
plt.plot(history.history["loss"], label="Training Loss")
plt.plot(history.history["val_loss"], label="Validation Loss")
plt.legend()
```

Out[69]:
```
<matplotlib.legend.Legend at 0x7bc0f8de8230>
```



The graph shows the *loss* (or the difference between the model's predictions and the actual data) for each epoch. There are several ways to calculate loss, and the method we have used is *mean absolute error*. There is a distinct loss value given for the training and the validation data. As a rule of thumb for this metric, 10% or less is an acceptable value for loss function. If it is not, you might want to refine the inner layer or the epochs.

Try varying the inner layer to different values between 1 and 16 and see how the loss function changes due to the changes.

Task 2.2

What happen to the loss function graphs as you vary the neurons in the inner layer (Embedding size)?

As you decrease the number of neurons in the inner layer, the slopes of the two loss functions flatten out and the two functions approach a flat constant line. As you increase the embedding size, the validation loss fluctuates more, and the training loss decreases significantly.

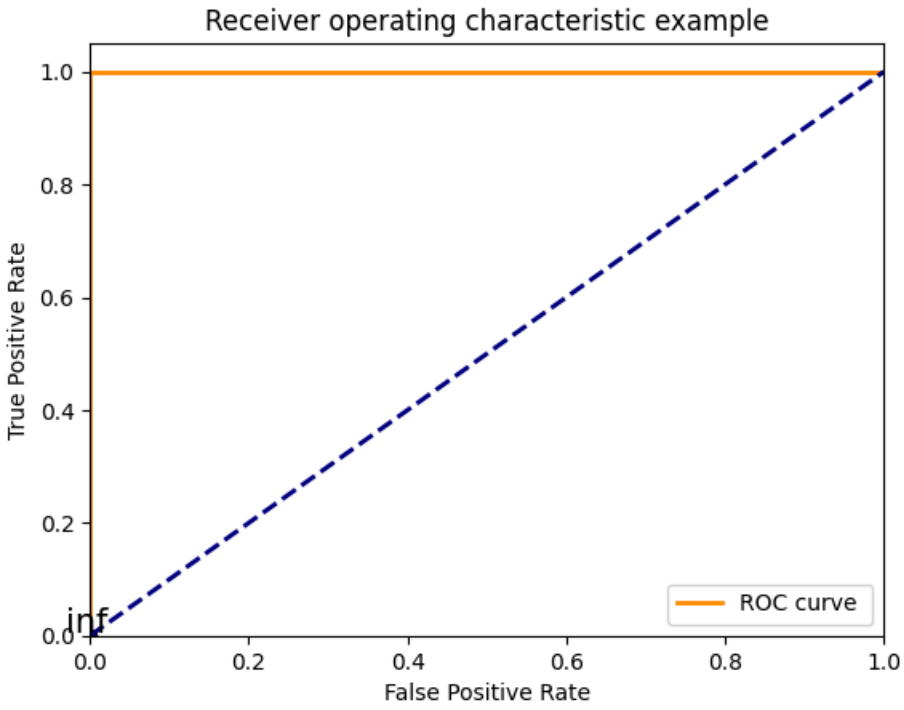
ROC and AUC Metrics

The Receiver Operating Characteristic (ROC) plots allows us to visualize the tradeoff between predicting anomalies as normal (false positives) and predicting normal data as an anomaly (false negative). Normal vibrations are labeled as `1` in this dataset but we have to flip them here to match the ROC curves expectations.

The ROC plot now has threshold values plotted on their corresponding points on the curve to aid in selecting a theshold for the application.

In [70]:
```
#Plotting True positive and false positive rate assessment
reconstructions = autoencoder(test_data)
loss = tf.keras.losses.mae(reconstructions, test_data)
fpr = []
tpr = []
#the test labels are flipped to match how the roc_curve function expects them.
flipped_labels = 1-test_labels
fpr, tpr, thresholds = roc_curve(flipped_labels, loss)
plt.figure()
lw = 2
plt.plot(fpr, tpr, color='darkorange',
```

```
         lw=lw, label='ROC curve ')
plt.plot([0, 1], [0, 1], color='navy', lw=lw, linestyle='--')
plt.xlim([0.0, 1.0])
plt.ylim([0.0, 1.05])
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('Receiver operating characteristic example')
plt.legend(loc="lower right")

# plot some thresholds
thresholds_every=20
thresholdsLength = len(thresholds)
colorMap=plt.get_cmap('jet', thresholdsLength)
for i in range(0, thresholdsLength, thresholds_every):
  threshold_value_with_max_four_decimals = str(thresholds[i])[:5]
  plt.scatter(fpr[i], tpr[i], c='black')
  plt.text(fpr[i] - 0.03, tpr[i] + 0.005, threshold_value_with_max_four_decimals, fontdict={'size': 15});

plt.show()
print(tpr)
```




```
[0.         0.00813008 1.         1.        ]
```

To pick the threshold that would give us the high true positive rate (TPR) and low false positive rate (FPR) we look at the 'knee' of the curve.

However, in some cases there may be an application constraint that requires a specific TPR or FPR, in which case we would have to move off of the 'knee' and sacrifice overall accuracy. In this case we might rather have false alarms than miss an abnormal vibration.

Now that we understand how to visualize the impact of the selected threshold, we calculate the area under the ROC curve (AUC).

This metric is very useful for evalutation of a specfic model design. Adjust the size of the encoding layer (EMBEDDING SIZE) in the autoencoder to maximize this metric.

In [71]:
```
roc_auc = auc(fpr, tpr)
print(roc_auc)
```

```
1.0
```

Task 2.3

How does the ROC changes as you vary the neurons in the inner layer (Embedding size)?

Using the above code, I was unable to change the ROC while varying the neurons in the inner layer. I tested the range 1-1000.

Picking a Threshold to Detect Anomalies

Detect anomalies by calculating whether the reconstruction loss is greater than a fixed threshold.

Try to maximize the accuracy, precision, and recall. Think about the application and the consequences of a false positive and a false negative.

Experiment by using different values (values labeled in black in the ROC graph) as a threshold below

More details on precision and recall

In [57]:
```
threshold = 0.061 #Assign a value labeled in black in the ROC graph  <-------------------------------------
print("Chosen Threshold: ", threshold)
```

```
Chosen Threshold:  0.061
```

In [58]:
```
def predict(model, data, threshold):
  reconstructions = model(data)
  loss = tf.keras.losses.mae(reconstructions, data)
  return tf.math.less(loss, threshold), loss

def print_stats(predictions, labels):
  print("Accuracy = {}".format(accuracy_score(labels, predictions)))
  print("Precision = {}".format(precision_score(labels, predictions)))
  print("Recall = {}".format(recall_score(labels, predictions)))
```

In [59]:
```
preds, scores = predict(autoencoder, test_data, threshold)
print_stats(preds, test_labels)
```

```
Accuracy = 0.5125
Precision = 0.0
Recall = 0.0
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 due to no predicted samples. Use `zero_division` parameter to control this behavior.
  _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
```

For most anomaly detection problems, accuracy is not enough to assess the power of detection of the model. Accuracy alone does not tell the complete story i.e. how often does the model flag an vibration as abnormal when it is indeed abnormal (true positive), abnormal when it is normal (false positive) normal when it is abnormal (false negative) and normal when it is indeed normal (true negative). Two important metrics can be applied to address these issues - precision aand recall.

Precision expresses the percentage of positive predictions that are correct and is calculated as (true positive / true positive + false positive).

Recall expresses the proportion of actual positives that were corrected predicted (true positive / true positive + false negative).

Depending on the use case, it may be desirable to optimize a model's performance for high precision or high recall. This tradeoff between precision and recall can be adjusted by the selection of a threshold (e.g. a low enough threshold will yield excellent recall but reduced precision). In addition, the Receiver Operating Characteristics (ROC) curve provides a

visual assessment of a model's skill (area under the curve - AUC) and is achieved by plotting the true positive rate against the false positive rate at various values of the threshold.

Task 2.4

Which threshold value provides the best metrics to refine the classification model? Explain why.

The area under the curve AUC on the Reciever Operating Characteristic ROC plot provides the best metrics to refine the classification model since it shows the clearest distinction between false positives and negatives. In addition, the precision and recall of the model can both be extrapolated from this value.

Please continue to Prelab 8.3 here.